

TEMA 1

PROYECTO 1: Importación Masiva mediante comando BULK INSERT con Manejo de Errores

1. Enunciado del ejercicio

La Oficina de Admisión y Registro de la UPLA provee un archivo plano delimitado por comas (EstudiantesMigracion.csv) con el padrón de alumnos egresados. Diseñe un procedimiento almacenado llamado Academico.SP_MigrarEstudiantesBulk que ejecute una importación masiva directa hacia la tabla Academico.Estudiante. El script debe omitir la primera fila del archivo por contener los encabezados, configurar el formato de codificación de caracteres en UTF-8 para preservar tildes/eñes, e incorporar un bloque transaccional robusto que asegure que si el archivo físico no se encuentra en la ruta del servidor (D:\MigracionUPLA\), la operación se cancele limpiamente.

2. Script de la solución en T-SQL

```
IF OBJECT_ID('Academico.SP_MigrarEstudiantesBulk', 'P') IS NOT NULL
    DROP PROCEDURE Academico.SP_MigrarEstudiantesBulk;
GO

CREATE PROCEDURE Academico.SP_MigrarEstudiantesBulk
    @RutaArchivo NVARCHAR(255)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Comando nativo de inserción en bloque para procesamiento de alto rendimiento
        DECLARE @ComandoSQL NVARCHAR(MAX);
        SET @ComandoSQL = N'
        BULK INSERT Academico.Estudiante
        FROM ''' + @RutaArchivo + N'''
        WITH (
            FIRSTROW = 2,          -- Omitir cabecera de columnas del CSV
            FIELDTERMINATOR = ',', -- Delimitador de campos
            ROWTERMINATOR = '\n',   -- Delimitador de registros
            CODEPAGE = '65001',     -- Forzar codificación UTF-8 para tildes y caracteres especiales
            TABLOCK                -- Bloqueo de tabla para optimizar la velocidad de inserción
        );';

        EXEC sp_executesql @ComandoSQL;

        COMMIT TRANSACTION;
        PRINT 'Importación masiva mediante BULK INSERT completada exitosamente.';
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        DECLARE @ErrMsg NVARCHAR(4000) = 'Fallo crítico en la carga masiva. Verifique la ruta del archivo o el formato de datos. Detalle: ' + ERROR_MESSAGE();
        RAISERROR(@ErrMsg, 16, 1);
    END CATCH
END
```

00 %
Messages
Commands completed successfully.

3. Justificación técnica de la solución aplicada

El comando BULK INSERT interactúa de forma directa con las páginas lógicas de datos de SQL Server a nivel de subsistema de almacenamiento, evitando el costo de procesamiento por fila individual que imponen los insertores iterativos tradicionales. Al definir CODEPAGE = '65001', se mapea el set de caracteres binarios hacia el estándar global UTF-8, garantizando que nombres autóctonos con eñes o acentos gráficos no sufran corrupciones de codificación física al almacenarse.

4. Explicación de las buenas prácticas utilizadas en el proyecto

Uso de la Sugerencia TABLOCK: Reduce drásticamente la generación de registros en el log de transacciones (.ldf) al habilitar la carga mínimamente registrada (Minimal Logging), lo cual eleva exponencialmente la velocidad de migración masiva.

Encapsulamiento Dinámico Seguro: Al validar la transacción global, se previene que la tabla quede en un estado parcialmente poblado si la lectura del archivo plano físico se interrumpe abruptamente en mitad de la transmisión de bytes.

PROYECTO 2: Carga de Datos Heterogéneos mediante la Función OPENROWSET

1. Enunciado del ejercicio

Para procesar un lote histórico excepcional de Resoluciones emitidas por el Consejo Universitario que se encuentra alojado transitoriamente en una hoja de cálculo externa de Microsoft Excel (Resoluciones2026.xlsx), implemente una consulta optimizada utilizando la función analítica nativa OPENROWSET. El script debe mapear y acoplar dinámicamente las columnas del libro de cálculo hacia la estructura del esquema Documento.Resolucion, forzando el uso del proveedor de base de datos OLE DB correspondiente.

2. Script de la solución en T-SQL

```
-- Habilitar características avanzadas de Ad-Hoc Distributed Queries en la instancia
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;
EXEC sp_configure 'Ad Hoc Distributed Queries', 1;
RECONFIGURE;
GO

-- Inserción directa mapeando el origen de datos heterogéneo
INSERT INTO Documento.Resolucion (
    IdExpediente, NumeroResolucion, FechaEmision, TipoResolucion, RutaDocumento, Activo, UsuarioModificacion
)
SELECT
    CAST(IdExpediente AS INT),
    CAST(NumeroResolucion AS VARCHAR(40)),
    CAST(FechaEmision AS DATETIME),
    CAST(TipoResolucion AS VARCHAR(30)),
    CAST(RutaDocumento AS VARCHAR(255)),
    1,
    'MIGRACION_ACE_OLEDB'
FROM OPENROWSET(
    'Microsoft.ACE.OLEDB.12.0',
    'Excel 12.0 Xml;Database=D:\MigracionUPLA\Resoluciones2026.xlsx;HDR=YES',
    'SELECT IdExpediente, NumeroResolucion, FechaEmision, TipoResolucion, RutaDocumento FROM [Hoja1$]'
);
GO
```

3. Justificación técnica de la solución aplicada

La función OPENROWSET actúa como un conector de consultas distribuidas ad-hoc que inicializa el proveedor OLE DB de Microsoft Access (ACE), permitiendo interpretar el formato XML nativo de los archivos abiertos de Excel (.xlsx) directo desde el motor relacional. Al incorporar HDR=YES, SQL Server trata automáticamente la primera fila del libro como nombres de columnas virtuales para operar la manipulación lógica de datos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Mapeo con Conversiones Explícitas (CAST): Debido a que Excel carece de tipos de datos relacionales estrictos y tiende a evaluar las columnas como variantes flotantes, el uso de CAST previene de forma nativa errores de truncamiento o incompatibilidades de tipos de datos DDL al persistir la información.
- Principio de Seguridad de Instancia: La apertura de consultas ad-hoc debe realizarse puntualmente para cargas específicas, manteniendo el aislamiento de red del servidor respecto a la ejecución de procesos externos maliciosos.

PROYECTO 3: Conversión Dinámica de Datos XML a Estructuras Relacionales

1. Enunciado del ejercicio

SUNEDU remite periódicamente lotes de validaciones institucionales en archivos de texto estructurados bajo formato XML. Desarrolle un script especializado en Transact-SQL que tome una variable de tipo XML conteniendo el padrón de diplomas aprobados, extraiga sus nodos internos jerárquicos (<Graduado>) utilizando los métodos nativos de consulta XML de SQL Server, y vuelque dicha información de forma tabular limpia hacia la tabla Documento.EmissionSunedu.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- Declaración de la variable XML con el lote de datos estructurados enviado por SUNEDU
DECLARE @XmlData XML = N'
<SuneduLote>
  <Graduado IdDiploma="1">
    <TicketEnvio>TK-2026-000182</TicketEnvio>
    <CodigoVerificacion>A93F48B2C71E5D6D8F9E0A1B2C3D4E5F</CodigoVerificacion>
    <EstadoSunedu>Inscrito_Oficial</EstadoSunedu>
  </Graduado>
  <Graduado IdDiploma="2">
    <TicketEnvio>TK-2026-000183</TicketEnvio>
    <CodigoVerificacion>7C8B9D0E1F2A3B4C5D6E7F8A9B0C1D2E</CodigoVerificacion>
    <EstadoSunedu>Inscrito_Oficial</EstadoSunedu>
  </Graduado>
</SuneduLote>;

-- Transformación de estructura XML jerárquica a filas y columnas relacionales
INSERT INTO Documento.EmissionSunedu (
  IdDiploma, FechaEnvio, TicketEnvio, CodigoVerificacion, EstadoSunedu, UsuarioModificacion
)
SELECT
  X.nodo.value('@IdDiploma', 'INT') AS IdDiploma,
  GETDATE() AS FechaEnvio,
  X.nodo.value('(TicketEnvio)[1]', 'VARCHAR(20)') AS TicketEnvio,
  X.nodo.value('(CodigoVerificacion)[1]', 'VARCHAR(64)') AS CodigoVerificacion,
  X.nodo.value('(EstadoSunedu)[1]', 'VARCHAR(30)') AS EstadoSunedu,
  'INTEGRACION_XML' AS UsuarioModificacion
FROM @XmlData.nodes('/SuneduLote/Graduado') AS X(nodo);
GO
```

3. Justificación técnica de la solución aplicada

El método nativo .nodes() fragmenta la estructura XML en memoria ram, generando un conjunto de filas lógicas temporales a partir del parámetro de ruta XPath especificado (/SuneduLote/Graduado). Complementariamente, la función miembro .value() compila de forma fuertemente tipada los atributos

(anteponiendo @) o subnodos de texto hacia los equivalentes exactos del motor relacional de base de datos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Uso del Índice Cardinalicio [1]: Al buscar subnodos, especificar explícitamente el índice numérico [1] le indica al optimizador de consultas que solo existirá una coincidencia singular por nodo, lo cual detiene el bucle interno de búsqueda del procesador y acelera el rendimiento de la consulta.
- Tipado Seguro Directo: Evita la necesidad de recurrir a la función obsoleta e ineficiente `sp_xml_preparedocument`, reduciendo drásticamente el consumo de memoria del servidor al no requerir el almacenamiento de árboles DOM completos en el búfer.

PROYECTO 4: Operaciones de Sincronización Avanzada con la Sentencia MERGE

1. Enunciado del ejercicio

La Oficina General de Asuntos Académicos actualiza de forma autónoma una tabla maestra de alumnos en un servidor intermedio. Diseñe un procedimiento almacenado llamado

`Academico.SP_SincronizarEstudiantesMaestro` que utilice la sentencia de coincidencia de conjuntos `MERGE` para sincronizar la tabla `Academico.Estudiante` (Tabla Destino) con la tabla externa de cambios. El script debe cumplir tres reglas de negocio atómicas en una sola instrucción:

1. Si el estudiante ya existe, actualizar sus datos personales e institucionales.
2. Si el estudiante es nuevo, insertarlo automáticamente.
3. Si un estudiante ya no figura en el padrón del servidor maestro, marcarlo con estado inactivo (Activo = 0) en nuestro sistema de Grados y Títulos.

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- Se asume la existencia previa de la tabla espejo o de carga intermedia
IF OBJECT_ID('Academico.EstudianteIntermedio', 'U') IS NULL
BEGIN
    CREATE TABLE Academico.EstudianteIntermedio (
        IdEstudiante      INT NOT NULL,
        CodigoEstudiante  VARCHAR(20) NOT NULL,
        Nombre            VARCHAR(50) NOT NULL,
        Apellido          VARCHAR(50) NOT NULL,
        Correo            VARCHAR(100) NOT NULL,
        Telefono          VARCHAR(20) NOT NULL,
        IdEscuela         INT NOT NULL
    );
END;
GO

IF OBJECT_ID('Academico.SP_SincronizarEstudiantesMaestro', 'P') IS NOT NULL
    DROP PROCEDURE Academico.SP_SincronizarEstudiantesMaestro;
GO

CREATE PROCEDURE Academico.SP_SincronizarEstudiantesMaestro
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Aplicación de Sincronización Atómica Completa mediante MERGE
        MERGE Academico.Estudiante AS Destino
        USING Academico.EstudianteIntermedio AS Origen
        ON (Destino.IdEstudiante = Origen.IdEstudiante)

        -- CASO 1: Coincidencia encontrada - Actualizar campos existentes
        WHEN MATCHED THEN
            UPDATE SET

```

3. Justificación técnica de la solución aplicada

La instrucción MERGE compila los tres subprocesos lógicos DML (INSERT, UPDATE, DELETE/UPDATE) en un único plan de ejecución unificado. Al evaluar el árbol de consultas en una sola pasada sobre las páginas indexadas de las tablas involucradas, reduce drásticamente el costo de lectura en disco en comparación con ejecutar consultas separadas mediante bloques condicionales IF EXISTS, optimizando el rendimiento general del motor.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Uso de Cláusulas Específicas de Destino (BY TARGET/BY SOURCE): Permite aislar con precisión matemática los registros huérfanos en ambos lados del conjunto, garantizando que el borrado lógico (Activo = 0) no altere la integridad referencial de los trámites históricos de la UPLA.
- Bloque Transaccional de Consistencia: Toda la operación se procesa de forma atómica. Si una sola fila presenta inconsistencias de datos, el flujo completo se revierte, protegiendo al sistema de estados de sincronización parciales.

PROYECTO 5: Consultas a Servidores Vinculados (Linked Servers) en Redes Distribuidas

1. Enunciado del ejercicio

Para conectar la base de datos de Grados y Títulos de la UPLA con el servidor central de la sede principal (SRV-HUANCAYO-CENTRAL), configure el acceso mediante un Servidor Vinculado (Linked Server). Posteriormente, diseñe una consulta transaccional distribuida utilizando nombres de objetos de cuatro

partes para cruzar los datos de las escuelas profesionales locales con el catálogo institucional centralizado de facultades.

2. Script de la solución en T-SQL

```
USE master;
GO

-- =====
-- CONFIGURACIÓN DEL SERVIDOR VINCULADO (LINKED SERVER)
-- =====
IF EXISTS (SELECT service_name FROM sys.servers WHERE name = 'SRV-HUANCAYO-CENTRAL')
BEGIN
    EXEC sp_dropserver 'SRV-HUANCAYO-CENTRAL', 'droplogins';
END;
GO

-- Crear el enlace lógico hacia la otra instancia de SQL Server de la UPLA
EXEC sp_addlinkedserver
    @server = N'SRV-HUANCAYO-CENTRAL',
    @srvproduct = N'SQL Server';
GO

-- Configurar las credenciales de seguridad seguras para el puente de red
EXEC sp_addlinkedsrvlogin
    @rmtsrvname = N'SRV-HUANCAYO-CENTRAL',
    @useself = N'FALSE',
    @locallogin = NULL,
    @rmtuser = N'UserIntegracionUpla',
    @rmtpassword = N'SeguridadCentral2026#';
GO

-- =====
-- CONSULTA TRANSACCIONAL DISTRIBUIDA (CUATRO PARTES)
-- =====
USE GradosTitulos;
GO

SELECT
    local_esc.IdEscuela,
    local_esc.Nombre AS EscuelaProfesionalLocal,
    remoto_fac.Nombre AS FacultadSedeCentral
-- Uso de nomenclatura estándar de 4 partes: [Servidor].[BaseDatos].[Esquema].[Objeto]
FROM Academico.Escuela local_esc
```

3. Justificación técnica de la solución aplicada

La configuración de un Linked Server registra un proveedor OLE DB de puente de red dentro de la instancia local de SQL Server. Al ejecutar la consulta utilizando el formato de cuatro partes (Servidor.BaseDatos.Esquema.Tabla), el optimizador local coordina con el motor remoto la ejecución del plan distribuido, transfiriendo únicamente el conjunto resultante optimizado a través de los sockets TCP de la red universitaria.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Mapeo de Seguridad Centralizado (@useself = N'FALSE'): Garantiza que los usuarios de la base de datos local no hereden privilegios de forma descontrolada en el servidor central, forzando la canalización de consultas a través de una cuenta de servicio de integración con permisos restringidos.
- Prevención de Caídas de Conexión: El script limpia instancias y mapeos huérfanos anteriores antes de redefinir el enlace, eliminando conflictos latentes de sesiones de red activas.

PROYECTO 6: Réplicas Remotas mediante la Función OPENQUERY para Optimización de Rendimiento

1. Enunciado del ejercicio

Al ejecutar consultas complejas de auditoría uniendo tablas locales con servidores remotos mediante la nomenclatura clásica de 4 partes, el rendimiento de la red se degrada debido al intercambio ineficiente de datos en bruto (Remote Scan). Optimice este proceso implementando una solución basada en la función passthrough OPENQUERY, enviando la instrucción de agregación directamente al servidor secundario para que sea procesada en el nodo remoto, retornando solo el resultado final resumido.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- =====
-- OPTIMIZACIÓN DE CONSULTA DISTRIBUIDA EN PASSTHROUGH CON OPENQUERY
-- =====

SELECT
    E.CodigoEstudiante,
    E.Nombre + ' ' + E.Apellido AS Egresado,
    ResRemota.NumeroResolucion,
    ResRemota.FechaEmision AS FechaAprobacionConsejo
FROM Academico.Estudiante E
INNER JOIN OPENQUERY([SRV-HUANCAYO-CENTRAL], '
    SELECT
        IdExpediente,
        NumeroResolucion,
        FechaEmision
    FROM GradosTitulosCentral.Documento.Resolucion
    WHERE Activo = 1 AND FechaEmision >= ''2026-01-01''
') AS ResRemota
ON E.IdEstudiante = ResRemota.IdExpediente
ORDER BY E.Apellido ASC;
GO
```

3. Justificación técnica de la solución aplicada

A diferencia del formato de cuatro partes donde el motor local descarga grandes volúmenes de datos remotos para operarlos en su propia memoria ram, OPENQUERY ejecuta una consulta de paso directo (Passthrough Query). Esto obliga al servidor remoto (SRV-HUANCAYO-CENTRAL) a procesar localmente los filtros WHERE y ordenamientos dentro de su propio motor de bases de datos, transmitiendo a través de la red únicamente el subconjunto de filas filtradas finales, reduciendo de manera drástica los tiempos de respuesta.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Reducción de Tráfico de Red (Payload): Al filtrar las fechas por cadenas de texto escapadas (''2026-01-01'') dentro del cuerpo de la consulta remota, se evita el envío innecesario de registros históricos obsoletos a través de los canales de la red institucional de la universidad.

- Aislamiento de Recursos de Memoria: Esta estrategia de ingeniería de datos protege el búfer del servidor local, evitando que consultas analíticas o de reportería institucional consuman los recursos destinados a las transacciones de los alumnos en tiempo real.

TEMA 2

PROYECTO 1: Inserción Automatizada de Estudiantes con Validación de Requisitos Básicos

1. Enunciado del ejercicio

La Universidad requiere automatizar el registro de alumnos en la tabla Academico.Estudiante. Diseñe un procedimiento almacenado llamado Academico.SP_RegistrarEstudiante que inserte un nuevo registro únicamente si se cumplen las siguientes condiciones:

El código de estudiante y el correo institucional no deben existir previamente en la base de datos.

La escuela profesional asociada (IdEscuela) debe ser válida y encontrarse activa. El procedimiento debe incluir un manejo de errores estructurado que cancele la operación ante cualquier incumplimiento de las reglas de negocio.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

IF OBJECT_ID('Academico.SP_RegistrarEstudiante', 'P') IS NOT NULL
    DROP PROCEDURE Academico.SP_RegistrarEstudiante;
GO

CREATE PROCEDURE Academico.SP_RegistrarEstudiante
    @CodigoEstudiante VARCHAR(20),
    @Nombre VARCHAR(50),
    @Apellido VARCHAR(50),
    @Correo VARCHAR(100),
    @Telefono VARCHAR(20),
    @IdEscuela INT,
    @UsuarioMod VARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        -- 1. Validar duplicidad de Código de Estudiante
        IF EXISTS (SELECT 1 FROM Academico.Estudiante WHERE CodigoEstudiante = @CodigoEstudiante)
        BEGIN
            RAISERROR('Regla de Negocio: El código de estudiante especificado ya se encuentra registrado.', 16, 1);
        END

        -- 2. Validar duplicidad de Correo Institucional
        IF EXISTS (SELECT 1 FROM Academico.Estudiante WHERE Correo = @Correo)
        BEGIN
            RAISERROR('Regla de Negocio: El correo electrónico ya pertenece a otro estudiante.', 16, 1);
        END

        -- 3. Validar existencia y estado de la Escuela Profesional
        IF NOT EXISTS (SELECT 1 FROM Academico.Escuela WHERE IdEscuela = @IdEscuela AND Activo = 1)
        BEGIN
            RAISERROR('Regla de Negocio: La escuela profesional especificada no existe o no está activa.', 16, 1);
        END

        -- 4. Inserción física en la base de datos
    END TRY
    BEGIN CATCH
        -- Manejo de errores
    END CATCH
END
```

3. Justificación técnica de la solución aplicada

El uso de condicionales IF EXISTS optimiza el motor de base de datos debido a que detiene el escaneo interno de los índices (Index Seek) en cuanto encuentra la primera fila coincidente, evitando lecturas lógicas de páginas innecesarias. Al emplear RAISERROR, se desvía el flujo de ejecución directo hacia el bloque CATCH, impidiendo que la sentencia INSERT se ejecute con datos inconsistentes.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Aislamiento de Reglas de Negocio: Se validan de forma independiente cada una de las restricciones lógicas antes de iniciar cualquier operación DML de escritura, asegurando mensajes de error descriptivos y precisos para el backend de la aplicación.
- Mantenimiento de Auditoría Obligatoria: El procedimiento exige los parámetros temporales y de usuario (GETDATE(), @UsuarioMod), cumpliendo con el estándar corporativo de rastreabilidad de modificaciones.

PROYECTO 2: Actualización de Estados de Expedientes con Control de Transiciones Válidas

1. Enunciado del ejercicio

Dentro del esquema Tramite, el flujo de un expediente no puede retroceder arbitrariamente ni saltarse etapas reguladas. Implemente un procedimiento almacenado llamado Tramite.SP_ActualizarEstadoExpediente que modifique el campo IdEstadoTramite de un expediente específico. El script debe comprobar de forma estricta que la transición solicitada cumpla con las políticas de la UPLA (por ejemplo: un expediente en estado 'Registrado' [1] solo puede pasar a 'Evaluación' [2] o 'Rechazado' [4], pero nunca directo a 'Aprobado' [3]).

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

IF OBJECT_ID('Tramite.SP_ActualizarEstadoExpediente', 'P') IS NOT NULL
    DROP PROCEDURE Tramite.SP_ActualizarEstadoExpediente;
GO

CREATE PROCEDURE Tramite.SP_ActualizarEstadoExpediente
    @IdExpediente INT,
    @NuevoEstadoTramite TINYINT,
    @UsuarioMod VARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @EstadoActual TINYINT;

    BEGIN TRY
        -- 1. Recuperar el estado actual del expediente
        SELECT @EstadoActual = IdEstadoTramite
        FROM Tramite.Expediente
        WHERE IdExpediente = @IdExpediente;

        IF @EstadoActual IS NULL
        BEGIN
            RAISERROR('Error: El expediente especificado no existe.', 16, 1);
        END

        -- 2. Validar matriz de transiciones permitidas utilizando lógica de control
        IF @EstadoActual = 1 AND @NuevoEstadoTramite NOT IN (2, 4) -- De Registrado a Evaluación o Rechazado
        BEGIN
            RAISERROR('Regla de Negocio: Transición de estado inválida para un expediente recién Registrado.', 16, 1);
        END

        IF @EstadoActual = 2 AND @NuevoEstadoTramite NOT IN (3, 4) -- De Evaluación a Aprobado o Rechazado
        BEGIN
            RAISERROR('Regla de Negocio: Un expediente en Evaluación solo puede cambiar a Aprobado o Rechazado.', 16, 1);
        END
    END TRY
    BEGIN CATCH
        RAISERROR('Error desconocido en SP_ActualizarEstadoExpediente.', 16, 1);
    END CATCH
END

```

3. Justificación técnica de la solución aplicada

Al almacenar temporalmente el estado en una variable local de memoria (@EstadoActual), el motor evita realizar múltiples lecturas repetitivas a la tabla física de expedientes durante las bifurcaciones lógicas. El operador condicional NOT IN evalúa eficientemente el conjunto explícito de transiciones válidas en tiempo de ejecución.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Prevención de Inconsistencias de Flujo de Trabajo: Evita que errores de software en la capa del cliente o modificaciones manuales por base de datos salten fases regulatorias críticas del trámite de Grados y Títulos.
- Control Preventivo ante Parámetros Inexistentes: Valida que el @IdExpediente devuelva un registro real antes de proceder con las evaluaciones lógicas, impidiendo ejecuciones en vacío.

PROYECTO 3: Eliminación Lógica Controlada de Documentos y Limpieza de Dependencias

1. Enunciado del ejercicio

Por normativas de auditoría, los registros históricos de diplomas y resoluciones en el esquema Documento no deben ser eliminados físicamente mediante sentencias DELETE. Diseñe un procedimiento almacenado llamado Documento.SP_EliminarLogicoDiploma que realice un borrado lógico estableciendo el campo Activo = 0. Asimismo, el procedimiento debe asegurar que si el diploma cuenta con registros asociados activos en subtablas dependientes (como Documento.LibroRegistro), estas relaciones cambien de estado de manera coordinada para mantener la integridad.

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

IF OBJECT_ID('Documento.SP_EliminarLogicoDiploma', 'P') IS NOT NULL
    DROP PROCEDURE Documento.SP_EliminarLogicoDiploma;
GO

CREATE PROCEDURE Documento.SP_EliminarLogicoDiploma
    @IdDiploma INT,
    @UsuarioMod VARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON; -- Asegura la reversión total ante fallos relacionales

    BEGIN TRY
        BEGIN TRANSACTION;

        -- 1. Validar existencia previa del registro
        IF NOT EXISTS (SELECT 1 FROM Documento.Diploma WHERE IdDiploma = @IdDiploma)
        BEGIN
            RAISERROR('Error: El diploma especificado no existe.', 16, 1);
        END

        -- 2. Propagación: Desactivar dependencias en cascada (Libro de Registro)
        UPDATE Documento.LibroRegistro
        SET EstadoRegistro = 'Anulado',
            UsuarioModificacion = @UsuarioMod
        WHERE IdDiploma = @IdDiploma;

        -- 3. Desactivación lógica de la entidad principal
        UPDATE Documento.Diploma
        SET Activo = 0,
            UsuarioModificacion = @UsuarioMod
        WHERE IdDiploma = @IdDiploma;

        COMMIT TRANSACTION;
        PRINT 'Eliminación lógica y desactivación de dependencias completada con éxito.';
    END TRY
END

```

3. Justificación técnica de la solución aplicada

El uso de una transacción explícita (BEGIN TRANSACTION) envuelve de forma atómica las dos instrucciones UPDATE. Esto garantiza que la actualización del diploma y la actualización de los estados en la tabla dependiente LibroRegistro ocurran de forma indivisible. Si una instrucción falla por un bloqueo o restricción foreign key, el motor ejecuta un ROLLBACK, regresando los datos a su estado original.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Preservación del Historial de Datos (No-Repudio): Sustituye la destrucción física de datos por cambios de estado lógicos (Activo = 0, 'Anulado'), permitiendo reconstruir auditorías institucionales pasadas requeridas por organismos de control.
- Uso Decidido de XACT_ABORT ON: Fuerza la cancelación inmediata de todo el lote transaccional si ocurre un error de ejecución en cualquiera de las consultas implícitas.

PROYECTO 4: Consulta Paginada de Expedientes Activos para Interfaces Web

1. Enunciado del ejercicio

Para optimizar el rendimiento del sistema intranet desarrollado en Java/NetBeans, se requiere construir una consulta de expedientes que soporte paginación de datos del lado del servidor. Diseñe un procedimiento almacenado llamado Tramite.SP_ListarExpedientesPaginados que devuelva las solicitudes de titulación activas ordenadas por fecha de ingreso. El procedimiento debe admitir como parámetros el número de página (@NumeroPagina) y la cantidad de filas por página (@RegistrosPorPagina).

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

IF OBJECT_ID('Tramite.SP_ListarExpedientesPaginados', 'P') IS NOT NULL
    DROP PROCEDURE Tramite.SP_ListarExpedientesPaginados;
GO

CREATE PROCEDURE Tramite.SP_ListarExpedientesPaginados
    @NumeroPagina INT,
    @RegistrosPorPagina INT
AS
BEGIN
    SET NOCOUNT ON;

    -- Validar parámetros base para evitar desbordamientos o divisiones por cero
    IF @NumeroPagina < 1 SET @NumeroPagina = 1;
    IF @RegistrosPorPagina < 1 SET @RegistrosPorPagina = 10;

    -- Consulta paginada optimizada mediante cláusulas ANSI SQL
    SELECT
        Ex.IdExpediente,
        E.CodigoEstudiante,
        E.Nombre + ' ' + E.Apellido AS Estudiante,
        Ex.FechaIngreso,
        Ex.IdEstadoTramite
    FROM Tramite.Expediente Ex
    INNER JOIN Academico.Estudiante E ON Ex.IdEstudiante = E.IdEstudiante
    WHERE Ex.Activo = 1
    ORDER BY Ex.FechaIngreso DESC
    OFFSET (@NumeroPagina - 1) * @RegistrosPorPagina ROWS
    FETCH NEXT @RegistrosPorPagina ROWS ONLY;
END;
GO
```

3. Justificación técnica de la solución aplicada

Las cláusulas avanzadas OFFSET y FETCH NEXT realizan una paginación nativa directamente sobre el árbol de índices de la base de datos de SQL Server. Esto evita la necesidad de transferir millones de registros hacia la memoria de la aplicación backend, disminuyendo la sobrecarga de la red y el consumo de memoria ram en el servidor.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Normalización Preventiva de Parámetros: El procedimiento corrige en caliente valores menores a uno (< 1), asegurando la estabilidad lógica del script ante peticiones erróneas del cliente web.
- Ordenamiento Predictivo Fijo: El uso de una cláusula ORDER BY explícita basada en claves de fecha asegura que el algoritmo de paginación devuelva resultados deterministas consistentes entre recargas de página.

PROYECTO 5: Reporte de Estadísticas de Sustentaciones con Uso de Parámetros de Salida (OUTPUT)

1. Enunciado del ejercicio

Diseñe un componente analítico llamado Evaluacion.SP_ObtenerMetricasSustentacion que evalúe el rendimiento académico del esquema de sustentaciones. El procedimiento debe examinar los actos

registrados dentro de un rango de fechas provisto y retornar tres valores métricos fundamentales utilizando parámetros de salida (OUTPUT):

El total de sustentaciones realizadas.

La cantidad de alumnos aprobados.

La nota promedio final obtenida en dicho periodo institucional.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

IF OBJECT_ID('Evaluacion.SP_ObtenerMetricasSustentacion', 'P') IS NOT NULL
    DROP PROCEDURE Evaluacion.SP_ObtenerMetricasSustentacion;
GO

CREATE PROCEDURE Evaluacion.SP_ObtenerMetricasSustentacion
    @FechaInicio          DATETIME,
    @FechaFin             DATETIME,
    @TotalSustentadas      INT OUTPUT,
    @TotalAprobados       INT OUTPUT,
    @PromedioGeneral      DECIMAL(4,2) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    -- 1. Obtener la cantidad total de sustentaciones en el rango indicado
    SELECT @TotalSustentadas = COUNT(*)
    FROM Evaluacion.Sustentacion
    WHERE FechaSustentacion BETWEEN @FechaInicio AND @FechaFin;

    -- 2. Obtener el total de alumnos aprobados (Promedio de notas rubros >= 11)
    SELECT @TotalAprobados = COUNT(*)
    FROM Evaluacion.Sustentacion
    WHERE FechaSustentacion BETWEEN @FechaInicio AND @FechaFin
    AND ((NotaExposicion + NotaDefensa + NotaConocimientos) / 3.0) >= 11.0;

    -- 3. Calcular la nota promedio general de todas las sustentaciones del periodo
    SELECT @PromedioGeneral = ISNULL(CAST(AVG((NotaExposicion + NotaDefensa + NotaConocimientos) / 3.0) AS DECIMAL(4,2)), 0.00)
    FROM Evaluacion.Sustentacion
    WHERE FechaSustentacion BETWEEN @FechaInicio AND @FechaFin;
END;
GO
```

3. Justificación técnica de la solución aplicada

Los parámetros marcados con la directiva OUTPUT devuelven los escalares de forma directa a las variables del hilo llamador de la sesión. Al concentrar las operaciones de agregación (COUNT, AVG) en una sola llamada y acotarlas mediante índices de fecha, SQL Server procesa la reducción de datos de forma compacta sin necesidad de estructurar conjuntos tabulares complejos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

Control de Resultados Nulos: Se emplea la función condicional ISNULL en el cálculo de promedios para garantizar que si un rango de fechas no devuelve registros, el parámetro retorne un valor numérico seguro (0.00) en lugar de un NULL que cause fallas en el código de aplicación Java.

Precisión Numérica Explícita: El casteo explícito a DECIMAL(4,2) asegura que el indicador de calidad académica mantenga la consistencia con el formato decimal del cuadro de mandos de Power BI de la UPLA.

PROYECTO 6: Búsqueda Avanzada de Tesis mediante Filtros Dinámicos Flexibles

1. Enunciado del ejercicio

Los usuarios de la biblioteca universitaria requieren buscar trabajos de investigación aplicando múltiples criterios opcionales (título, año de publicación o escuela profesional). Desarrolle un procedimiento almacenado robusto llamado `Tramite.SP_BuscarTrabajosFlexibles` que implemente lógica de filtros dinámicos. Si un parámetro se pasa como nulo (NULL), el procedimiento debe ignorar dicha restricción, devolviendo las coincidencias exactas y parciales combinadas.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

IF OBJECT_ID('Tramite.SP_BuscarTrabajosFlexibles', 'P') IS NOT NULL
    DROP PROCEDURE Tramite.SP_BuscarTrabajosFlexibles;
GO

CREATE PROCEDURE Tramite.SP_BuscarTrabajosFlexibles
    @Titulo    VARCHAR(255) = NULL,
    @Anio      INT = NULL,
    @IdEscuela INT = NULL
AS
BEGIN
    SET NOCOUNT ON;

    -- Búsqueda flexible utilizando lógica booleana parametrizada (Short-Circuit Evaluation)
    SELECT
        TI.IdTrabajo,
        TI.Titulo,
        TI.FechaRegistro,
        E.Nombre AS EscuelaProfesional,
        TI.Estado
    FROM Tramite.TrabajoInvestigacion TI
    INNER JOIN Academico.Estudiante Est ON TI.IdEstudiante = Est.IdEstudiante
    INNER JOIN Academico.Escuela E      ON Est.IdEscuela = E.IdEscuela
    WHERE (@Titulo IS NULL OR TI.Titulo LIKE '%' + @Titulo + '%')
        AND (@Anio IS NULL OR YEAR(TI.FechaRegistro) = @Anio)
        AND (@IdEscuela IS NULL OR Est.IdEscuela = @IdEscuela)
        AND TI.Estado <> 'Anulado';
END;
GO
```

3. Justificación técnica de la solución aplicada

La estructura de predicados lógicos basados en la premisa (`@Parametro IS NULL OR Columna = @Parametro`) aprovecha el cortocircuito lógico del optimizador de SQL Server. Si el parámetro es enviado como NULL, la primera sección de la instrucción OR se evalúa como verdadera, provocando que el motor ignore por completo el costo de evaluar la columna física indexada, manteniendo un plan de ejecución altamente eficiente y reutilizable.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso Inteligente de Parámetros por Defecto:** Se configuran los valores iniciales de entrada como = NULL. Esto permite invocar al procedimiento almacenado enviando únicamente los criterios conocidos, reduciendo la complejidad del código del lado del cliente.
- **Aislamiento de Registros de Descarte:** El filtro estático `TI.Estado <> 'Anulado'` limpia automáticamente los reportes públicos de la biblioteca de documentos no aptos para consulta, asegurando la consistencia de la información institucional expuesta.

TEMA 3

PROYECTO 1: Vista Compleja para el Consolidado de Condiciones de Egresados

1. Enunciado del ejercicio

La Oficina de Asuntos Académicos de la UPLA requiere un reporte consolidado en tiempo real que determine la situación de los estudiantes que iniciaron trámites de titulación. Diseñe una vista compleja llamada Academico.V_CondicionEgresadoCompleta que una las entidades de Estudiantes, Escuelas Profesionales y Facultades. La vista debe calcular dinámicamente si el alumno cuenta con un expediente de trámite aprobado o si su condición se encuentra aún en estado "Pendiente de Validación".

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

IF OBJECT_ID('Academico.V_CondicionEgresadoCompleta', 'V') IS NOT NULL
    DROP VIEW Academico.V_CondicionEgresadoCompleta;
GO

CREATE VIEW Academico.V_CondicionEgresadoCompleta
AS
SELECT
    E.IdEstudiante,
    E.CodigoEstudiante,
    E.Nombre + ' ' + E.Apellido AS Estudiante,
    EP.Nombre AS EscuelaProfesional,
    F.Nombre AS Facultad,
    ISNULL(EX.IdExpediente, 0) AS NroExpedienteAsociado,
    -- Evaluación lógica del estado del egresado basada en la existencia del trámite
    CASE
        WHEN EX.IdEstadoTramite = 3 THEN 'Apto para Titulación'
        WHEN EX.IdEstadoTramite = 1 OR EX.IdEstadoTramite = 2 THEN 'Trámite en Evaluación'
        ELSE 'Pendiente de Validación'
    END AS CondicionInstitucional
FROM Academico.Estudiante E
INNER JOIN Academico.Escuela EP ON E.IdEscuela = EP.IdEscuela
INNER JOIN Academico.Facultad F ON EP.IdFacultad = F.IdFacultad
LEFT JOIN Tramite.Expediente EX ON E.IdEstudiante = EX.IdEstudiante AND EX.Activo = 1;
GO
```

3. Justificación técnica de la solución aplicada

Esta solución utiliza un acoplamiento externo por la izquierda (LEFT JOIN) para asegurar que ningún estudiante quede excluido del reporte analítico final si es que aún no ha iniciado un expediente en la tabla Tramite.Expediente. La evaluación condicional CASE se ejecuta directo en el procesador de consultas, lo que mantiene la integridad referencial y respeta la Tercera Forma Normal (3FN) al no almacenar datos derivados estáticos en las tablas físicas.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Uso Inteligente de Funciones de Nulidad (ISNULL): Reemplaza de forma segura las referencias vacías en columnas numéricas por un valor neutral cero (0), facilitando el parseo de datos en el backend de NetBeans.

- Encapsulación de Uniones Múltiples: Al aislar los tres JOIN complejos dentro de la vista, se simplifica el desarrollo de la interfaz de usuario, permitiendo invocar el reporte mediante un comando estructurado simple (SELECT * FROM Academico.V_CondicionEgresadoCompleta).

PROYECTO 2: Función en Línea Multiinstrucción (TVF) para la Carga de Proyectos por Asesor

1. Enunciado del ejercicio

Para balancear la carga de trabajo del cuerpo docente de la UPLA, se solicita una función con valor de tabla de múltiples instrucciones (Table-Valued Function - TVF) llamada

Tramite.F_ObtenerCargaProyectosDocente. Esta función debe recibir como parámetro el identificador de un docente (@IdDocente) y retornar una estructura tabular bien definida que contenga los títulos de las tesis asignadas, la escuela profesional solicitante y los días transcurridos desde que se registró la investigación.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

IF OBJECT_ID('Tramite.F_ObtenerCargaProyectosDocente', 'TF') IS NOT NULL
    DROP FUNCTION Tramite.F_ObtenerCargaProyectosDocente;
GO

CREATE FUNCTION Tramite.F_ObtenerCargaProyectosDocente (
    @IdDocente INT
)
RETURNS @TablaCarga TABLE (
    IdTrabajo          INT NOT NULL,
    TituloTesis        VARCHAR(255) NOT NULL,
    EscuelaProfesional VARCHAR(100) NOT NULL,
    DiasEnProceso      INT NOT NULL,
    EstadoTramite      VARCHAR(40) NOT NULL
)
AS
BEGIN
    -- Poblar la tabla de retorno con la lógica transaccional combinada
    INSERT INTO @TablaCarga
    SELECT
        TI.IdTrabajo,
        TI.Titulo,
        EP.Nombre,
        DATEDIFF(DAY, TI.FechaRegistro, GETDATE()),
        TI.Estado
    FROM Tramite.TrabajoInvestigacion TI
    INNER JOIN Tramite.Asesoramiento A ON TI.IdTrabajo = A.IdTrabajo
    INNER JOIN Academico.Estudiante E ON TI.IdEstudiante = E.IdEstudiante
    INNER JOIN Academico.Escuela EP ON E.IdEscuela = EP.IdEscuela
    WHERE A.IdDocente = @IdDocente AND A.Activo = 1;

    RETURN;
END;
GO
```

3. Justificación técnica de la solución aplicada

Las funciones de tabla de multiinstrucciones actúan creando una variable de tabla interna temporal controlada en el entorno de ejecución de SQL Server. A diferencia de las vistas tradicionales, esta función permite incrustar variables operativas complejas en el filtro WHERE y realizar cálculos algorítmicos sobre vectores de tiempo como DATEDIFF en la memoria volátil del servidor de forma aislada.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Definición de Esquema Fuerte en Retorno: La tabla devuelta se autodeclara con tipos de datos e integridades estrictas individuales, lo que previene errores de desalineación en el mapeo de objetos del software cliente.
- Control de Vigencia en Relaciones: Al filtrar explícitamente mediante la condición A.Activo = 1, la función omite asesorías pasadas o desvinculadas, entregando métricas vigentes y reales del docente evaluado.

PROYECTO 3: Función Escalar Determinista para Validación de Formato de Códigos Universitarios

1. Enunciado del ejercicio

Con el propósito de mitigar fallos de digitación humana en los módulos de mesa de partes, implemente una función escalar llamada Catalogo.F_ValidarCodigoUPLA. Esta función recibirá una cadena de texto representando un código estudiantil y evaluará si cumple rígidamente con las políticas institucionales (longitud exacta de 10 caracteres y estructura compuesta por el año de ingreso y caracteres numéricos), devolviendo un indicador de bandera tipo BIT (1 si es correcto, 0 si es inválido).

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

IF OBJECT_ID('Catalogo.F_ValidarCodigoUPLA', 'FN') IS NOT NULL
    DROP FUNCTION Catalogo.F_ValidarCodigoUPLA;
GO

CREATE FUNCTION Catalogo.F_ValidarCodigoUPLA (
    @CodigoEstudiante VARCHAR(30)
)
RETURNS BIT
AS
BEGIN
    DECLARE @Resultado BIT = 0;

    -- Validar longitud exacta de 10 caracteres y coincidencia con patrón de expresión regular
    IF LEN(@CodigoEstudiante) = 10 AND @CodigoEstudiante LIKE '[2][0][0-2][0-9]%'
    BEGIN
        -- Verificar si el resto de los componentes de la cadena son puramente numéricos
        IF ISNUMERIC(@CodigoEstudiante) = 1
            SET @Resultado = 1;
    END

    RETURN @Resultado;
END;
GO
```

3. Justificación técnica de la solución aplicada

Al resolver la lógica mediante operaciones nativas de cadenas de texto (LEN, LIKE posicional), la función opera de manera determinista pura, procesando los caracteres secuencialmente a alta velocidad en el pipeline del CPU del servidor de base de datos, evitando recurrir a subconsultas relacionales pesadas a tablas de catálogo.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Diseño para Reutilización en Capas: Al retornar un tipo de dato primitivo estándar (BIT), este componente puede acoplarse con total versatilidad tanto dentro de restricciones de verificación de tablas (CHECK CONSTRAINTS) como en capas condicionales de procedimientos almacenados.
- Inmunidad ante Inyecciones de Código: Al procesar el parámetro mediante operadores puramente lógicos de comparación de patrones, se neutralizan intentos de desbordamiento de búfer o alteraciones maliciosas en los formularios del sistema.

PROYECTO 4: Trigger para Validación de Requisitos Previos en Expedientes (Integridad de Negocio)

1. Enunciado del ejercicio

El Reglamento General de Grados y Títulos de la UPLA dictamina que un alumno no puede dar de alta un expediente para la obtención del Título Profesional (IdTipoTramite = 2) si es que previamente no cuenta con un expediente aprobado para el grado de Bachiller. Diseñe un disparador de base de datos llamado Tramite.TR_ValidarRequisitoBachiller asociado a la tabla Tramite.Expediente que aborte cualquier intento de inserción que vulnere esta directiva de calidad académica.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

IF OBJECT_ID('Tramite.TR_ValidarRequisitoBachiller', 'TR') IS NOT NULL
    DROP TRIGGER Tramite.TR_ValidarRequisitoBachiller;
GO

CREATE TRIGGER Tramite.TR_ValidarRequisitoBachiller
ON Tramite.Expediente
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    -- Validar si se está intentando registrar un trámite de Titulación Profesional (Tipo 2)
    IF EXISTS (
        SELECT 1
        FROM inserted i
        WHERE i.IdTipoTramite = 2
    )
    BEGIN
        -- Comprobar si el alumno asociado de la fila insertada carece de Bachiller Aprobado
        IF EXISTS (
            SELECT 1
            FROM inserted i
            WHERE NOT EXISTS (
                SELECT 1
                FROM Tramite.Expediente E_Bachiller
                WHERE E_Bachiller.IdEstudiante = i.IdEstudiante
                AND E_Bachiller.IdTipoTramite = 1 -- Grado de Bachiller
                AND E_Bachiller.IdEstadoTramite = 3 -- Estado Aprobado / Concedido
                AND E_Bachiller.Activo = 1
            )
        )
        BEGIN
            -- Cancelar la transacción entera de inserción y emitir alerta regulatoria
            ROLLBACK TRANSACTION;
            RAISERROR('Control Institucional: El estudiante no cuenta con el grado de Bachiller formalmente aprobado. Inserción denegada.', 16, 1);
        END
    END
END
```

3. Justificación técnica de la solución aplicada

El disparador intercepta el lote transaccional post-ejecución preliminar (AFTER INSERT), teniendo acceso inmediato a la tabla lógica virtual inserted. El uso de subconsultas correlacionadas inversas mediante NOT

EXISTS permite validar de forma relacional cruzada que la precondition reglamentaria exista en la base de datos; al invocar el comando ROLLBACK TRANSACTION, se anula de inmediato el puntero de almacenamiento, dejando la tabla base en su estado limpio original.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Enfoque Orientado a Bloques de Conjuntos: El trigger evita el uso ineficiente de bucles iterativos o cursores en el código del disparador, procesando los lotes completos de registros en una sola operación atómica.
- Protección de Datos en Origen: La regla de negocio se blindo a nivel físico en el motor. De esta forma, si un operador administrativo intenta saltarse la validación desde un gestor externo (como otra consola SQL), el motor rechazará el comando automáticamente, impidiendo inconsistencias de datos.

PROYECTO 5: Trigger de Auditoría en Tiempo Real para el Control de Calificaciones de Sustentación

1. Enunciado del ejercicio

Con el fin de mitigar riesgos de alteración fraudulenta de calificaciones en las actas de sustentación de tesis de pregrado, implemente un disparador de auditoría estricto llamado Evaluacion.TR_AuditarCambioNotas. Cada vez que ocurra una modificación (UPDATE) en las columnas de calificaciones de la tabla Evaluacion.Sustentacion, el trigger debe capturar los valores anteriores y nuevos e insertarlos automáticamente en la tabla histórica de control.

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- Asegurar la existencia previa de la tabla histórica de auditoría de notas
IF OBJECT_ID('Evaluacion.HistorialCalificaciones', 'U') IS NULL
BEGIN
    CREATE TABLE Evaluacion.HistorialCalificaciones (
        IdLogNota          INT IDENTITY(1,1) NOT NULL,
        IdSustentacion      INT NOT NULL,
        ColumnaAlterada     VARCHAR(50) NOT NULL,
        NotaAnterior        DECIMAL(4,2) NOT NULL,
        NotaNueva           DECIMAL(4,2) NOT NULL,
        FechaRegistro       DATETIME NOT NULL DEFAULT GETDATE(),
        UsuarioAuditoria    VARCHAR(100) NOT NULL,
        CONSTRAINT PK_HistorialCalificaciones PRIMARY KEY (IdLogNota)
    );
END;
GO

IF OBJECT_ID('Evaluacion.TR_AuditarCambioNotas', 'TR') IS NOT NULL
    DROP TRIGGER Evaluacion.TR_AuditarCambioNotas;
GO

CREATE TRIGGER Evaluacion.TR_AuditarCambioNotas
ON Evaluacion.Sustentacion
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- 1. Capturar modificaciones en Nota de Exposición
    IF UPDATE(NotaExposicion)
    BEGIN
        INSERT INTO Evaluacion.HistorialCalificaciones (IdSustentacion, ColumnaAlterada, NotaAnterior, NotaNueva, UsuarioAuditoria)
        SELECT i.IdSustentacion, 'NotaExposicion', d.NotaExposicion, i.NotaExposicion, SYSTEM_USER
        FROM inserted i
        INNER JOIN deleted d ON i.IdSustentacion = d.IdSustentacion
        WHERE i.NotaExposicion <> d.NotaExposicion;
    END
END

```

3. Justificación técnica de la solución aplicada

Al utilizar la directiva analítica incorporada IF UPDATE(NombreColumna), el trigger optimiza su ejecución evaluando la máscara de bits de la transacción para determinar si los campos específicos fueron modificados en el comando DML original. Al realizar un INNER JOIN directo entre las tablas virtuales inserted (datos nuevos) y deleted (datos históricos), el motor extrae de forma limpia la delta exacta de variación de datos de las calificaciones.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Uso de Firmas de Auditoría Imborrables (SYSTEM_USER): Almacena de manera transparente la cuenta de sistema o credencial autenticada en SQL Server que procesó el comando de actualización, asegurando la validez jurídica del rastro técnico de la auditoría.
- Filtro de Descarte de No-Cambio: La cláusula de validación WHERE i.NotaExposicion <> d.NotaExposicion asegura que solo se registren registros en el log histórico si el valor numérico realmente varió, previniendo el almacenamiento redundante de datos idénticos.

PROYECTO 6: Trigger de Sincronización en Cascada para la Activación Lógica de Escuelas Profesionales

1. Enunciado del ejercicio

Cuando la alta dirección de la UPLA decide suspender o reactivar administrativamente una Facultad entera debido a procesos de reorganización institucional, dicha acción debe propagarse automáticamente a todas las Escuelas Profesionales que dependen de ella. Diseñe un trigger llamado Academico.TR_SincronizarEstadoFacultad que automatice la actualización en cascada del campo Activo sobre la tabla Academico.Escuela en respuesta a una modificación en la tabla Academico.Facultad.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

IF OBJECT_ID('Academico.TR_SincronizarEstadoFacultad', 'TR') IS NOT NULL
    DROP TRIGGER Academico.TR_SincronizarEstadoFacultad;
GO

CREATE TRIGGER Academico.TR_SincronizarEstadoFacultad
ON Academico.Facultad
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Validar si el campo crítico de estado operacional sufrió modificaciones
    IF UPDATE(Activo)
    BEGIN
        -- Propagar de manera inmediata el cambio a todas las escuelas dependientes
        UPDATE E
        SET E.Activo = i.Activo,
            E.FechaModificacion = GETDATE(),
            E.UsuarioModificacion = 'SYNC_CASCADA_TRIGGER'
        FROM Academico.Escuela E
        INNER JOIN inserted i ON E.IdFacultad = i.IdFacultad
        INNER JOIN deleted d ON i.IdFacultad = d.IdFacultad
        WHERE i.Activo <> d.Activo; -- Actuar únicamente ante variaciones reales de estado
    END
END;
GO
```

3. Justificación técnica de la solución aplicada

La arquitectura de actualización utiliza un bloque de unión basado en conjuntos (UPDATE FROM JOIN) que asocia directamente la tabla de producción física Academico.Escuela con los estados transitorios de la tabla virtual inserted. Esta técnica le permite al motor de la base de datos reescribir de manera masiva y en un solo ciclo de reloj los bits de estado de múltiples registros hijos concurrentes, sin necesidad de recurrir a costosos cursores o disparadores iterativos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Prevención de Bucles Infinitos de Ejecución: El disparador valida la condición de cambio real ($i.Activo \neq d.Activo$), evitando que el trigger gatille actualizaciones redundantes en cascada que degraden la memoria del servidor de la universidad.
- Trazabilidad del Proceso: Al sellar el campo UsuarioModificacion con una firma fija explicativa ('SYNC_CASCADA_TRIGGER'), se clarifica a los auditores técnicos del sistema que la alteración de la escuela fue operada automáticamente por el motor y no por una acción manual directa del usuario operativo.

TEMA 4

PROYECTO 1: Control de Transacciones Atómicas mediante TRY...CATCH y XACT_ABORT

1. Enunciado del ejercicio

Al registrar la aprobación final de un expediente de titulación en la UPLA, se deben realizar múltiples inserciones y actualizaciones en simultáneo (crear el registro del diploma, actualizar el estado del trámite y archivar el libro de registro). Diseñe un procedimiento almacenado llamado Documento.SP_AprobarEgresadoCompleto que implemente una transacción atómica robusta. Si ocurre un error en cualquier punto del lote DML, se debe revertir toda la operación (ROLLBACK) de manera inmediata y segura utilizando las propiedades nativas de manejo de excepciones.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

IF OBJECT_ID('Documento.SP_AprobarEgresadoCompleto', 'P') IS NOT NULL
    DROP PROCEDURE Documento.SP_AprobarEgresadoCompleto;
GO

CREATE PROCEDURE Documento.SP_AprobarEgresadoCompleto
    @IdExpediente INT,
    @NumeroDiploma VARCHAR(30),
    @NumLibro VARCHAR(20),
    @NumFolio VARCHAR(20),
    @UsuarioMod VARCHAR(50)
AS
BEGIN
    -- Configuración crítica para abortar transacciones de forma inmediata ante errores graves
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    DECLARE @IdEstudiante INT;

    BEGIN TRY
        -- 1. Recuperar información del estudiante asociado al expediente
        SELECT @IdEstudiante = IdEstudiante
        FROM Trámite.Expediente
        WHERE IdExpediente = @IdExpediente AND Activo = 1;

        IF @IdEstudiante IS NULL
        BEGIN
            RAISERROR('Error: El expediente especificado no existe o no está activo.', 16, 1);
        END

        -- Inicio explícito de la zona transaccional atómica
        BEGIN TRANSACTION;

        -- 2. Insertar el Diploma Oficial correspondientemente
        DECLARE @IdDiploma INT;
```

3. Justificación técnica de la solución aplicada

La combinación de la directiva SET XACT_ABORT ON con la función de estado XACT_STATE() garantiza un control transaccional robusto. Si SQL Server experimenta un error en tiempo de ejecución (como una violación de clave foránea o pérdida transitoria de comunicación con el disco), XACT_ABORT marca inmediatamente la transacción como no confirmable (uncommittable transaction), delegando el control al bloque CATCH donde se ejecuta un ROLLBACK seguro, evitando estados inconsistentes o confirmaciones parciales.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Uso Adecuado de SCOPE_IDENTITY(): Captura el último identificador auto-generado dentro del mismo ámbito del hilo de ejecución actual, garantizando la inserción correcta de las claves primarias en las relaciones secundarias sin interferencias de otros usuarios concurrentes.

- Manejo Descriptivo de Alertas: Limpia los búferes y encapsula las excepciones del motor en un formato semántico estandarizado, permitiendo un consumo limpio de mensajes de error desde la capa de la aplicación en NetBeans.

PROYECTO 2: Implementación de Nivel de Aislamiento SNAPSHOT para Consultas Analíticas Concurrentes

1. Enunciado del ejercicio

Los analistas de la UPLA ejecutan consultas pesadas en Power BI para calcular el promedio de notas de sustentaciones, bloqueando temporalmente las tablas e impidiendo que los docentes ingresen nuevas calificaciones. Para solucionar esta contención de recursos, habilite el nivel de aislamiento basado en versiones de filas de bases de datos (SNAPSHOT) para Grados y Títulos, y diseñe una consulta de reporte que lea de forma consistente el estado de los datos sin solicitar bloqueos compartidos de lectura.

2. Script de la solución en T-SQL

```
USE master;
GO

-- =====
-- FASE 1: HABILITAR EL CONTROL DE VERSIONES DE FILAS (SNAPSHOT)
-- =====
ALTER DATABASE GradosTitulos SET ALLOW_SNAPSHOT_ISOLATION ON;
GO

-- =====
-- FASE 2: CONSULTA ANALÍTICA EN AISLAMIENTO SNAPSHOT
-- =====
USE GradosTitulos;
GO

-- Configurar la sesión actual para utilizar aislamiento por Snapshot
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;

BEGIN TRANSACTION;

SELECT
    E.Nombre AS EscuelaProfesional,
    COUNT(S.IdSustentacion) AS TotalSustentaciones,
    AVG((S.NotaExposicion + S.NotaDefensa + S.NotaConocimientos) / 3.0) AS PromedioPeriodo
FROM Evaluacion.Sustentacion S
INNER JOIN Tramite.TrabajoInvestigacion TI ON S.IdTrabajo = TI.IdTrabajo
INNER JOIN Academico.Estudiante Est ON TI.IdEstudiante = Est.IdEstudiante
INNER JOIN Academico.Escuela E ON Est.IdEscuela = E.IdEscuela
GROUP BY E.Nombre;

COMMIT TRANSACTION;
GO
```

3. Justificación técnica de la solución aplicada

Al activar ALLOW_SNAPSHOT_ISOLATION, SQL Server instruye al motor a almacenar copias de las filas modificadas transaccionalmente dentro de la base de datos del sistema tempdb (almacén de versiones). Cuando la consulta analítica inicia bajo el nivel SNAPSHOT, lee la versión de las filas tal como estaban al inicio de su transacción. Al no solicitar bloqueos compartidos (Shared Locks - S), las operaciones

concurrentes de inserción y modificación de notas se ejecutan en paralelo sin experimentar retrasos ni contención de recursos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Consistencia de Lectura Absoluta: A diferencia del aislamiento READ UNCOMMITTED (que permite lecturas sucias), SNAPSHOT garantiza que los datos presentados en el reporte de Power BI sean transaccionalmente consistentes y correspondan a confirmaciones reales en el disco.
- Mitigación de Bloqueos en Tablas Transaccionales: Elimina por completo los cuellos de botella generados por la concurrencia entre consultas analíticas y operaciones operativas de registro de notas escolares.

PROYECTO 3: Uso Práctico de Puntos de Guardado mediante SAVE TRANSACTION

1. Enunciado del ejercicio

Durante el proceso secuencial de migración de expedientes académicos, se requiere evaluar de manera individual el cumplimiento de los rubros documentales del estudiante. Diseñe un script en T-SQL que procese un lote estructurado utilizando puntos de retorno condicionales (SAVE TRANSACTION). El flujo operativo debe confirmar la inserción de los datos básicos del expediente, pero si ocurre un fallo menor durante la carga del archivo digital de la tesis, la transacción debe retornar al punto de control previo, permitiendo salvar la inserción inicial.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

SET NOCOUNT ON;
SET XACT_ABORT OFF; -- Se establece en OFF de manera obligatoria para permitir rollback parcial a puntos de guardado

BEGIN TRANSACTION;

BEGIN TRY
    -- 1. Operación Primaria Obligatoria: Inserción base del trámite
    INSERT INTO Tramite.Expediente (
        IdEstudiante, IdTipoTramite, IdEstadoTramite, FechaIngreso, Activo, FechaModificacion, UsuarioModificacion
    ) VALUES (
        1, 1, 1, GETDATE(), 1, GETDATE(), 'OPERADOR_MIGRACION'
    );

    DECLARE @NuevoIdExpediente INT = SCOPE_IDENTITY();

    -- =====
    -- CREACIÓN DEL PUNTO DE RETORNO INTERMEDIO (SAVEPOINT)
    -- =====
    SAVE TRANSACTION PuntoExpedienteBase;

    BEGIN TRY
        -- 2. Operación Secundaria Opcional: Registro adjunto del documento físico o digital
        -- Simulación intencional de un error relacional insertando un IdTrabajo inexistente o forzando un error
        INSERT INTO Tramite.TrabajoInvestigacion (
            IdEstudiante, Titulo, Resumen, UrlArchivo, FechaRegistro, Estado, FechaModificacion, UsuarioModificacion
        ) VALUES (
            99999, 'Título de Investigación Inconsistente', 'Resumen...', 'http://upla.edu.pe/tesis/err.pdf', GETDATE(), 'Registrado', GETDATE(), 'OPERADOR_MIGRACION'
        );
    END TRY
    BEGIN CATCH
        -- Captura del fallo secundario: Retornar parcialmente al estado guardado
        PRINT 'Aviso: Error detectado en la carga de la tesis. Aplicando Rollback parcial al PuntoExpedienteBase...';
        ROLLBACK TRANSACTION PuntoExpedienteBase;
    END CATCH
END TRY
```

3. Justificación técnica de la solución aplicada

La instrucción SAVE TRANSACTION clona marcadores del puntero de transacciones dentro del log logístico (.ldf) sin liberar los bloqueos exclusivos asignados a los recursos modificados. Al invocar un ROLLBACK TRANSACTION NombrePunto, el motor revierte únicamente las instrucciones DML ejecutadas después de

la declaración del marcador, permitiendo continuar el bloque de ejecución principal hasta el comando COMMIT definitivo.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Resiliencia ante Errores no Críticos: Permite que fallas accesorias (como la falta de un documento digital adjunto o un campo opcional nulo) no descarten el progreso de las inserciones principales de datos de expedientes ya validadas.
- Control de Modos de Operación: Requiere establecer de manera explícita SET XACT_ABORT OFF, asegurando que SQL Server no interrumpa de manera automática la sesión y permita evaluar el control controlado de errores a nivel de código.

PROYECTO 4: Simulación y Configuración de Prioridades de Deadlock (Prioridad de Interbloqueo)

1. Enunciado del ejercicio

Cuando dos procesos intentan modificar los mismos recursos cruzados en órdenes inversos, el motor experimenta un interbloqueo (Deadlock). Diseñe un script estructurado que prepare y configure la prioridad de interbloqueo de una sesión mediante SET DEADLOCK_PRIORITY. Establezca que el proceso de auditoría institucional sea considerado como la víctima prioritaria predeterminada (LOW) ante colisiones con el proceso transaccional de aprobación de diplomas, garantizando la continuidad de los servicios estudiantiles de la UPLA.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- =====
-- PROCESO SIMULADO: ASIGNACIÓN DE PRIORIDAD BAJA (PROCESO VÍCTIMA)
-- =====

-- Configurar la sesión actual para tener baja prioridad ante conflictos de interbloqueo
SET DEADLOCK_PRIORITY LOW; -- Valores admitidos: LOW (ó -5), NORMAL (0), HIGH (5)
GO

BEGIN TRANSACTION;

-- Paso 1: Bloquear recurso A (Modificar tabla de Diplomas)
UPDATE Documento.Diploma
SET UsuarioModificacion = 'PROCESO_VÍCTIMA_LOW'
WHERE IdDiploma = 1;

-- Simulación de retraso de tiempo para propiciar la concurrencia cruzada del Proceso B
WAITFOR DELAY '00:00:05';

-- Paso 2: Intentar bloquear recurso B (Modificar tabla de Expedientes que posee el Proceso B)
UPDATE Tramite.Expediente
SET UsuarioModificacion = 'PROCESO_VÍCTIMA_LOW'
WHERE IdExpediente = 1;

COMMIT TRANSACTION;
GO
```

3. Justificación técnica de la solución aplicada

Cuando el monitor de interbloqueos de SQL Server (Deadlock Monitor Thread) detecta una dependencia circular cerrada entre dos o más hilos de ejecución, evalúa el valor de prioridad de las sesiones involucradas. Al establecer explícitamente SET DEADLOCK_PRIORITY LOW en la consulta analítica, el motor elige automáticamente dicha sesión como la víctima del deadlock (Deadlock Victim), matando su proceso y ejecutando un rollback automático, permitiendo que la sesión transaccional con prioridad superior complete sus operaciones en disco.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Protección del Núcleo Operativo del Negocio: Asegura que los procesos analíticos secundarios o de reportes cedan el control ante colisiones con los hilos críticos de registro de trámites de los alumnos en tiempo real.
- Manejo Recomendado en Aplicaciones: Al implementar prioridades, la aplicación cliente afectada debe contar con lógica de reintento (Retry Logic) para volver a enviar la consulta de forma automatizada tras ser finalizada por el motor.

PROYECTO 5: Monitoreo y Diagnóstico de Bloqueos mediante Vistas de Administración Dinámica (DMV)

1. Enunciado del ejercicio

Ante reportes constantes de congelamiento de pantalla en la intranet universitaria, el departamento de redes necesita identificar en tiempo real qué usuario o terminal está reteniendo bloqueos exclusivos y causando retrasos en el sistema. Desarrolle una consulta avanzada de diagnóstico utilizando las vistas de administración dinámica (sys.dm_tran_locks y sys.dm_exec_requests) que devuelva el ID de sesión del bloqueador, la tabla afectada y el texto de la consulta T-SQL que está ocasionando la contención de recursos.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- =====
-- SCRIPT DE AUDITORÍA Y MONITOREO DE CONTENCIÓN Y BLOQUEOS (DMV)
-- =====

SELECT
    tl.request_session_id AS ID_Sesion_Bloqueada,
    wt.blocking_session_id AS ID_Sesion_Bloqueadora,
    DB_NAME(tl.resource_database_id) AS BaseDatos,
    OBJECT_NAME(tl.resource_associated_entity_id) AS TablaAfectada,
    tl.request_mode AS Tipo_Bloqueo_Solicitado,
    der.status AS Estado_Peticion,
    dest.text AS Consulta_SQL_Ejecutada
FROM sys.dm_tran_locks tl
INNER JOIN sys.dm_os_waiting_tasks wt ON tl.lock_owner_address = wt.resource_address
INNER JOIN sys.dm_exec_requests der ON wt.blocking_session_id = der.session_id
CROSS APPLY sys.dm_exec_sql_text(der.sql_handle) AS dest
WHERE tl.resource_database_id = DB_ID('GradosTitulos');
GO
```

3. Justificación técnica de la solución aplicada

Las Vistas de Administración Dinámica (DMV) leen directamente las estructuras de control internas del programador de hilos de SQL Server. La tabla sys.dm_tran_locks expone el mapa activo de recursos reservados y solicitados en memoria, mientras que la función de tabla por cruce aplicativa sys.dm_exec_sql_text extrae el texto plano SQL desde la caché de planes de ejecución utilizando el puntero binario (sql_handle), permitiendo aislar la causa raíz del bloqueo.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso de Funciones No Invasivas:** Las consultas sobre las DMV se procesan a nivel de catálogos del sistema sin requerir bloqueos de lectura sobre las tablas de producción de los alumnos de la UPLA, garantizando la inocuidad del monitoreo.
- **Diagnóstico de Causa Raíz Eficiente:** Provee al Administrador de Base de Datos (DBA) la información exacta de la consulta ofensiva y el identificador de proceso (SPID) para tomar acciones correctivas inmediatas (como optimizar índices o terminar la sesión mediante un comando KILL).

PROYECTO 6: Implementación del Bloqueo Explícito de Filas mediante la Sugerencia UPDLOCK

1. Enunciado del ejercicio

En los días de alta concurrencia de sustentaciones, dos secretarios académicos pueden intentar asignar el mismo número correlativo de diploma a expedientes distintos. Para blindar este proceso crítico y evitar condiciones de carrera, diseñe una consulta transaccional que implemente la sugerencia avanzada de bloqueo de filas UPDLOCK combinada con HOLDLOCK. De esta manera, al momento de consultar la disponibilidad de un folio o diploma, el registro se reservará de manera exclusiva hasta que concluya toda la transacción, impidiendo lecturas simultáneas de otras terminales.

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- =====
-- TRANSACCIÓN CON BLOQUEO PREVENTIVO EXPLICITO DE FILAS
-- =====
BEGIN TRANSACTION;

BEGIN TRY
    -- Consultar y bloquear la fila del diploma específico de forma preventiva
    -- UPDLOCK: Toma bloqueos de actualización en lugar de compartidos
    -- HOLDLOCK: Mantiene el bloqueo estricto hasta que finalice la transacción entera (aislamiento Serializable)
    DECLARE @Disponibilidad VARCHAR(40);

    SELECT @Disponibilidad = EstadoRegistro
    FROM Documento.LibroRegistro WITH (UPDLOCK, HOLDLOCK)
    WHERE NumLibro = 'L-2026-PRE' AND NumFolio = 'F-00150';

    -- Evaluar regla de negocio en base al estado de reserva de la fila bloqueada
    IF @Disponibilidad = 'Conformado'
    BEGIN
        RAISERROR('Alerta: El folio y libro solicitado ya se encuentra impreso y validado.', 16, 1);
    END

    -- Proceder con la actualización física segura sabiendo que nadie más puede leer o alterar la fila
    UPDATE Documento.LibroRegistro
    SET EstadoRegistro = 'Conformado',
        FechaModificacion = GETDATE(),
        UsuarioModificacion = 'CONCURRENCIA_UPDLOCK'
    WHERE NumLibro = 'L-2026-PRE' AND NumFolio = 'F-00150';

    COMMIT TRANSACTION;
    PRINT 'Reserva y actualización ejecutada bajo control concurrente UPDLOCK exitoso.';
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
    BEGIN
        ROLLBACK TRANSACTION;
    END
END

```

3. Justificación técnica de la solución aplicada

Por defecto, las sentencias SELECT aplican bloqueos compartidos (S) que se liberan tan pronto como los datos terminan de leerse. Al inyectar la sugerencia WITH (UPDLOCK, HOLDLOCK), SQL Server adquiere de forma inmediata bloqueos de actualización (U) y los retiene hasta el fin del bloque transaccional. Esto impide de manera absoluta que cualquier otra conexión adquiera un bloqueo similar o intente modificar el registro en paralelo, eliminando por completo los problemas de condiciones de carrera y actualizaciones perdidas.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Prevención Activa de Colisiones: Asegura la consistencia en la numeración oficial de los grados académicos expedidos por la UPLA sin incurrir en la costosa sobrecarga de elevar el nivel de aislamiento a toda la base de datos de manera global.
- Control de Ámbito Restringido: El bloqueo explícito se aplica estrictamente sobre las filas indexadas que cumplen el criterio del filtro WHERE, minimizando el impacto sobre otras consultas concurrentes de trámites.

TEMA 5

PROYECTO 1: Creación de un Índice Non-Clustered Buscado con Columnas Incluidas (Covering Index)

1. Enunciado del ejercicio

La intranet de la universidad ejecuta constantemente una consulta en la pantalla principal para listar a los estudiantes (Nombre, Apellido y Correo) filtrando por una escuela profesional específica (IdEscuela). Actualmente la consulta realiza un escaneo completo de la tabla (Table Scan o Clustered Index Scan). Diseñe e implemente un índice no agrupado cubriente (Covering Index) que optimice esta búsqueda, eliminando la necesidad de realizar operaciones de búsqueda en la tabla base (Key Lookup).

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- 1.1. Eliminar el índice si existiera previamente para garantizar la idempotencia del script
IF EXISTS (SELECT * FROM sys.indexes WHERE name = 'IX_Estudiante_IdEscuela_Covering' AND object_id = OBJECT_ID('Academico.Estudiante'))
BEGIN
    DROP INDEX IX_Estudiante_IdEscuela_Covering ON Academico.Estudiante;
END;
GO

-- 1.2. Crear el índice no agrupado optimizado con columnas incluidas (Covering Index)
CREATE NONCLUSTERED INDEX IX_Estudiante_IdEscuela_Covering
ON Academico.Estudiante (IdEscuela)
INCLUDE (Nombre, Apellido, Correo)
WITH (FILLFACTOR = 80, ONLINE = ON);
GO

-- 1.3. Consulta de verificación que aprovecha el índice al 100%
SELECT IdEscuela, Nombre, Apellido, Correo
FROM Academico.Estudiante
WHERE IdEscuela = 3; -- Ejemplo: Escuela de Ingeniería de Sistemas
GO
```

3. Justificación técnica de la solución aplicada

Al definir IdEscuela como la clave del índice, SQL Server organiza las páginas lógicas en un árbol B+ balanceado basándose en ese criterio. La inclusión de los campos Nombre, Apellido y Correo mediante la cláusula INCLUDE inyecta estos datos directamente en los nodos hoja del índice. De este modo, cuando el motor procesa el filtro WHERE, encuentra toda la información requerida por el SELECT en el mismo subconjunto del índice, transformando un escaneo pesado en un salto de índice inmediato (*Index Seek*) sin generar el costo de CPU de un *Key Lookup*.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso de Factores de Relleno Ajustados (FILLFACTOR = 80):** Deja un 20% de espacio libre en las páginas de datos del índice para absorber nuevas inserciones de alumnos de la UPLA sin provocar fragmentaciones prematuras por división de páginas (*Page Splits*).
- **Construcción en Línea (ONLINE = ON):** Permite que la estructura se cree en el servidor sin bloquear las transacciones activas de los usuarios en la base de datos de producción.

PROYECTO 2: Implementación de un Índice Filtrado (Filtered Index) para Registros Activos

1. Enunciado del ejercicio

El volumen de expedientes de titulación históricos en la universidad es masivo, pero el sistema web de mesa de partes solo interactúa con los expedientes que se encuentran actualmente en estado "Activo" o en trámite (Activo = 1). Crear un índice tradicional sobre toda la tabla desperdicia espacio en disco y memoria. Diseñe e implemente un Índice Filtrado (Filtered Index) que indexe exclusivamente los expedientes activos, reduciendo el tamaño del almacenamiento del índice y acelerando las búsquedas operativas.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

IF EXISTS (SELECT * FROM sys.indexes WHERE name = 'IX_Expediente_FiltroActivos' AND object_id = OBJECT_ID('Tramite.Expediente'))
BEGIN
    DROP INDEX IX_Expediente_FiltroActivos ON Tramite.Expediente;
END;
GO

-- 2.1. Creación del Índice Filtrado con predicado condicional estático
CREATE NONCLUSTERED INDEX IX_Expediente_FiltroActivos
ON Tramite.Expediente (IdEstudiante, IdTipoTramite)
WHERE Activo = 1
WITH (FILLFACTOR = 85);
GO

-- 2.2. Consulta operativa que se beneficia del índice filtrado
SELECT IdEstudiante, IdTipoTramite
FROM Tramite.Expediente
WHERE Activo = 1 AND IdEstudiante = 450;
GO
```

3. Justificación técnica de la solución aplicada

Un índice filtrado incorpora una cláusula condicional WHERE en su definición DDL. Esto restringe las filas que ingresan al árbol B+ del índice. El optimizador de SQL Server reconoce este predicado y, cuando una consulta solicita datos que coinciden exactamente con la condición del filtro (`Activo = 1`), utiliza directamente este subconjunto indexado compacto, reduciendo las lecturas lógicas e incrementando la eficiencia de la memoria caché.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Ahorro de Almacenamiento y Mantenimiento:** Al omitir el 90% de los datos históricos e inactivos de la universidad, el tamaño físico del archivo de base de datos se mantiene optimizado y los procesos automáticos de mantenimiento de índices consumen menos recursos de I/O.
- **Precisión en la Clave del Índice:** Se definen los campos clave en función de las uniones y filtros más frecuentes de los trámites, asegurando la selectividad de la estructura.

PROYECTO 3: Optimización mediante Índices de Columnas Agrupadas (Clustered Columnstore Index)

1. Enunciado del ejercicio

La Oficina de Planificación y Calidad Universitaria requiere procesar consultas analíticas sobre millones de registros históricos de emisión de diplomas y resoluciones institucionales para generar estadísticas anuales. Las consultas tradicionales por filas saturan el hardware del servidor. Implemente una optimización masiva migrando o añadiendo una estructura de Índice de Columnas Agrupadas (Clustered Columnstore Index) sobre una tabla histórica de métricas para acelerar de forma drástica las agregaciones analíticas.

2. Script de la solución en T-SQL

```
USE GradosYTítulos,
GO

-- 3.1. Crear una tabla de réplica histórica analítica para la demostración de Columnstore
IF OBJECT_ID('Documento.HistorialEmisionesAnalitico', 'U') IS NOT NULL
    DROP TABLE Documento.HistorialEmisionesAnalitico;
GO

CREATE TABLE Documento.HistorialEmisionesAnalitico (
    IdHistorial INT IDENTITY(1,1) NOT NULL,
    IdDiploma INT NOT NULL,
    AnioEmision INT NOT NULL,
    MontoTramite DECIMAL(10,2) NOT NULL,
    CodigoEscuela INT NOT NULL
);
GO

-- 3.2. Aplicar el Índice de Almacenamiento por Columnas Agrupadas (Columnstore)
CREATE CLUSTERED COLUMNSTORE INDEX CCI_HistorialEmisionesAnalitico
ON Documento.HistorialEmisionesAnalitico;
GO

-- 3.3. Consulta analítica de agregación masiva optimizada por el motor en modo columna
SELECT
    CodigoEscuela,
    SUM(MontoTramite) AS TotalRecaudado,
    COUNT(IdDiploma) AS CantidadDiplomas
FROM Documento.HistorialEmisionesAnalitico
GROUP BY CodigoEscuela;
GO
```

3. Justificación técnica de la solución aplicada

A diferencia de los índices tradicionales basados en filas (*Rowstore*), un *Columnstore Index* comprime los datos por columnas y los almacena en segmentos lógicos independientes (*Segments*). Cuando se ejecuta una consulta analítica de agregación (SUM, COUNT), el motor de SQL Server lee únicamente los bytes de los segmentos de las columnas involucradas en la operación, ignorando por completo el resto de campos. Esto reduce la transferencia de datos desde el almacenamiento hacia la memoria en un factor de hasta 10x, habilitando además el modo de ejecución por lotes (*Batch Mode*).

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Segmentación Analítica:** Este tipo de indexación se reserva exclusivamente para entornos de carga de datos masivos o tablas de reportería histórica, ya que está diseñado para optimizar lecturas analíticas en lugar de operaciones transaccionales frecuentes fila por fila (INSERT unitarios).
- **Alta Tasa de Compresión Nativa:** Reduce el costo total de propiedad (TOC) de la infraestructura de hardware de la universidad al requerir menos espacio en los discos de almacenamiento del servidor.

PROYECTO 4: Análisis de Planes de Ejecución y Diagnóstico de Operaciones SARGable

1. Enunciado del ejercicio

Un error común en las consultas desarrolladas para el sistema de Grados y Títulos es la aplicación de funciones sobre las columnas indexadas en la cláusula WHERE (por ejemplo, WHERE YEAR(FechaEmision) = 2026), lo cual destruye la capacidad del motor de usar los índices. Diseñe un script de diagnóstico que

reescriba una consulta no SARGable (Search Argumentable) a una estructura SARGable pura, comparando y justificando el cambio mediante el análisis del plan de ejecución.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- Asegurar un índice sobre la columna de fecha para la demostración
IF NOT EXISTS (SELECT * FROM sys.indexes WHERE name = 'IX_Diploma_FechaEmision' AND object_id = OBJECT_ID('Documento.Diploma'))
BEGIN
    CREATE NONCLUSTERED INDEX IX_Diploma_FechaEmision ON Documento.Diploma (FechaEmision);
END;
GO

-- =====
-- CASO MALO: CONSULTA NO SARGABLE (Provoca Index Scan / Alto costo de CPU)
-- =====
SELECT NumeroDiploma, FechaEmision
FROM Documento.Diploma
WHERE YEAR(FechaEmision) = 2026; -- La función oculta el valor al índice
GO

-- =====
-- CASO OPTIMIZADO: CONSULTA SARGABLE PURA (Provoca Index Seek / Alta velocidad)
-- =====
SELECT NumeroDiploma, FechaEmision
FROM Documento.Diploma
WHERE FechaEmision >= '2026-01-01' AND FechaEmision < '2027-01-01'; -- Rangos limpios planos
GO
```

3. Justificación técnica de la solución aplicada

Cuando se envuelve una columna en una función como YEAR(), el optimizador de SQL Server no puede calcular de forma anticipada los límites del rango del árbol del índice. Como resultado, se ve obligado a evaluar la función fila por fila en toda la tabla, desencadenando un escaneo completo de índices (*Index Scan*). Al reescribir la consulta utilizando comparadores de rango planos (>= y <), los valores se vuelven explícitos. El motor calcula las constantes inmediatamente y ejecuta una búsqueda de alto rendimiento (*Index Seek*).

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Evitar Funciones en Predicados:** Mantener las columnas de filtrado completamente limpias de funciones lógicas, matemáticas o de conversión en las cláusulas WHERE y JOIN.
- **Uso de Literales de Fecha Estándar:** Formatear las cadenas de fecha bajo el estándar ISO universal (YYYY-MM-DD) para evitar ambigüedades de interpretación regional por parte del motor de base de datos.

PROYECTO 5: Mantenimiento y Actualización de Estadísticas del Motor de Base de Datos

1. Enunciado del ejercicio

Debido al constante flujo de inserciones y modificaciones en el padrón de alumnos egresados, las estadísticas de distribución de datos almacenadas por el optimizador de consultas tienden a desactualizarse. Esto provoca que el motor elija planes de ejecución obsoletos o ineficientes. Diseñe un script administrativo automatizado para la UPLA que realice el mantenimiento y actualización completa de las estadísticas con muestreo total (FULLSCAN) de las tablas del esquema Academico.

2. Script de la solución en T-SQL


```
USE GradosTitulos;
GO

-- 5.1. Actualizar estadísticas de una tabla específica con muestreo completo (Fullscan)
PRINT 'Iniciando actualización de estadísticas para Academico.Estudiante...';
UPDATE STATISTICS Academico.Estudiante WITH FULLSCAN, COLUMNS;
GO

-- 5.2. Actualizar estadísticas de toda la base de datos mediante procedimiento nativo del sistema
PRINT 'Actualizando de forma global todas las estadísticas de la base de datos...';
EXEC sp_updatestats;
GO
```

3. Justificación técnica de la solución aplicada

Las estadísticas son objetos internos ocultos que almacenan histogramas de distribución del volumen y la densidad de los datos de las columnas. El optimizador de consultas recurre a ellos para estimar la cantidad de filas resultantes (*Cardinality Estimation*) de una consulta y elegir entre operaciones como un *Hash Match* o un *Nested Loops*. Al forzar `WITH FULLSCAN`, SQL Server lee el 100% de las filas de la tabla para reconstruir el histograma con precisión matemática absoluta, eliminando estimaciones erróneas.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Mantenimiento Programado Preventivo:** Ejecutar estos scripts de mantenimiento durante ventanas de bajo tráfico de usuarios, asegurando que las variaciones masivas de datos tras los periodos de matrícula o graduación no afecten el rendimiento diario.
- **Optimización de Recursos del Servidor:** Al mantener actualizados los metadatos de las estadísticas, se previene la compilación ineficiente de planes que saturan la memoria ram o el procesador del servidor universitario.

PROYECTO 6: Diagnóstico de Consultas Lentas mediante el uso de Extended Events (Eventos Extendidos)

1. Enunciado del ejercicio

Para reemplazar las antiguas trazas pesadas de SQL Server Profiler (depreciadas por Microsoft), implemente una sesión de Eventos Extendidos (Extended Events) de nivel de ingeniería. Esta sesión de monitoreo debe capturar en segundo plano, y de manera asíncrona, cualquier consulta ejecutada en la base de datos GradosTitulos cuya duración supere los 500 milisegundos, registrando el texto SQL exacto y la cantidad de lecturas lógicas consumidas.

2. Script de la solución en T-SQL

```

USE master;
GO

-- 6.1. Eliminar la sesión de eventos extendidos si ya existiera en la instancia
IF EXISTS (SELECT * FROM sys.server_event_sessions WHERE name = 'MonitoreoConsultasLentasUPLA')
BEGIN
    ALTER EVENT SESSION MonitoreoConsultasLentasUPLA ON SERVER STATE = STOP;
    DROP EVENT SESSION MonitoreoConsultasLentasUPLA ON SERVER STATE;
END;
GO

-- 6.2. Crear la sesión de eventos extendidos enfocada en rendimiento
CREATE EVENT SESSION MonitoreoConsultasLentasUPLA ON SERVER
ADD EVENT sqlserver.sql_statement_completed(
    ACTION(sqlserver.sql_text, sqlserver.database_name, sqlserver.client_hostname)
    WHERE (duration >= 500000 AND sqlserver.database_name = N'GradosTitulos') -- Duración en microsegundos (500 ms)
)
ADD TARGET package0.event_file(SET filename=N'D:\BaseDatos2026\Backups\ConsultasLentasUPLA.xel')
WITH (MAX_MEMORY=4096 KB, EVENT_RETENTION_MODE=ALLOW_SINGLE_EVENT_LOSS, MAX_DISPATCH_LATENCY=30 SECONDS, TRACK_CAUSALITY=OFF, STARTUP_STATE=OFF);
GO

-- 6.3. Activar la sesión de monitoreo en el servidor
ALTER EVENT SESSION MonitoreoConsultasLentasUPLA ON SERVER STATE = START;
GO

```

3. Justificación técnica de la solución aplicada

Extended Events (XEvents) es un sistema general de monitoreo de rendimiento altamente escalable integrado en el núcleo de SQL Server. Consume una cantidad mínima de recursos en comparación con las herramientas de traza tradicionales. Al configurar un destino de tipo archivo binario (event_file con extensión .xel), el motor escribe los eventos de forma asíncrona utilizando buffers de memoria intermedios controlados (MAX_MEMORY), garantizando que el rastreo de consultas lentas no penalice el rendimiento de las transacciones operativas.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Filtros de Captura Precisos:** Al acotar la sesión mediante condiciones explícitas (duration >= 500000 y nombre de base de datos), se evita la recolección masiva e innecesaria de telemetría de consultas rápidas, optimizando el espacio del almacenamiento local.
- **Estrategia Moderna de Diagnóstico (Tuning):** Provee al equipo de TI de la UPLA una bitácora técnica confiable para identificar de manera científica los módulos de software o reportes exactos que requieren una optimización de índices o reescritura de código T-SQL.

TEMA 6

PROYECTO 1: Creación de un Rol de Base de Datos Personalizado (Database Role) para Trámites

1. Enunciado del ejercicio

La Oficina de Grados y Títulos de la UPLA requiere restringir el acceso a la base de datos para los nuevos asistentes administrativos. Diseñe e implemente un rol de base de datos personalizado llamado RolAsistenteTramites. Este rol debe tener permisos exclusivos de lectura (SELECT) e inserción (INSERT) únicamente sobre las tablas del esquema Tramite y Documento, impidiendo explícitamente cualquier acción de modificación estructural (DDL) o eliminación de registros físicos.

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- 1.1. Eliminar el rol si existiera previamente para garantizar la reejecución limpia
IF DATABASE_PRINCIPAL_ID('RolAsistenteTramites') IS NOT NULL
BEGIN
    DROP ROLE RolAsistenteTramites;
END;
GO

-- 1.2. Crear el nuevo rol personalizado en la base de datos
CREATE ROLE RolAsistenteTramites;
GO

-- 1.3. Conceder permisos declarativos específicos a nivel de esquemas
GRANT SELECT, INSERT ON SCHEMA::Tramite TO RolAsistenteTramites;
GRANT SELECT, INSERT ON SCHEMA::Documento TO RolAsistenteTramites;

-- 1.4. Denegar explícitamente permisos de eliminación para robustecer la seguridad
DENY DELETE ON SCHEMA::Tramite TO RolAsistenteTramites;
DENY DELETE ON SCHEMA::Documento TO RolAsistenteTramites;
PRINT 'Rol de base de datos RolAsistenteTramites configurado con éxito.';
GO

```

3. Justificación técnica de la solución aplicada

El uso de roles de base de datos personalizados implementa el principio de seguridad de privilegios mínimos. Al aplicar las instrucciones GRANT a nivel de contenedor lógico (SCHEMA::) en lugar de hacerlo objeto por objeto, se simplifica la administración de la seguridad; cualquier tabla nueva que se agregue a los esquemas Tramite o Documento heredará automáticamente estas restricciones, reduciendo la sobrecarga operativa del administrador.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso del Enfoque Preventivo (DENY):** En SQL Server, una denegación explícita (DENY) tiene precedencia absoluta sobre cualquier permiso concedido indirectamente por otros roles del sistema, bloqueando la destrucción de datos.
- **Abstracción del Acceso Directo:** Evita asociar usuarios individuales directamente a los objetos relacionales, canalizando los permisos a través de un rol intermedio fácilmente gestionable.

PROYECTO 2: Implementación de Seguridad a Nivel de Filas (Row-Level Security - RLS)

1. Enunciado del ejercicio

Por políticas de confidencialidad institucional, los coordinadores académicos de la UPLA solo deben tener permitido visualizar los registros de los estudiantes que pertenezcan a su propia escuela profesional. Desarrolle una solución basada en Seguridad a Nivel de Filas (RLS) que filtre automáticamente los resultados de la tabla Academico.Estudiante basándose en el contexto de la sesión o el nombre del usuario conectado.

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- 2.1. Limpieza preventiva de políticas y funciones RLS anteriores
IF EXISTS (SELECT * FROM sys.security_policies WHERE name = 'FiltroSeguridadEstudiantes')
    DROP SECURITY POLICY Academico.FiltroSeguridadEstudiantes;
GO
IF OBJECT_ID('Academico.F_FiltroEscuelaUsuario', 'IF') IS NOT NULL
    DROP FUNCTION Academico.F_FiltroEscuelaUsuario;
GO

-- 2.2. Crear la función de predicado en línea (Inline Table-Valued Function)
CREATE FUNCTION Academico.F_FiltroEscuelaUsuario(@IdEscuela INT)
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN SELECT 1 AS Resultado
WHERE
    -- El Administrador del Sistema (sa) puede ver todo sin restricciones
    USER_NAME() = 'dbo'
    -- O el usuario de la sesión coincide con el nombre de la escuela (ej: 'E_SISTEMAS' mapeado al IdEscuela 3)
    OR (USER_NAME() = 'E_SISTEMAS' AND @IdEscuela = 3)
    OR (USER_NAME() = 'E_CIVIL' AND @IdEscuela = 1);
GO

-- 2.3. Crear y activar la política de seguridad RLS
CREATE SECURITY POLICY Academico.FiltroSeguridadEstudiantes
ADD FILTER PREDICATE Academico.F_FiltroEscuelaUsuario(IdEscuela)
ON Academico.Estudiante
WITH (STATE = ON);
PRINT 'Política de Seguridad a Nivel de Filas (RLS) activada correctamente.';
GO

```

3. Justificación técnica de la solución aplicada

Row-Level Security (RLS) intercepta el motor de ejecución agregando el predicado de la función en línea (Schemabinding) como un filtro implícito no modificable dentro de la cláusula WHERE de cualquier consulta enviada a la tabla. Esto ocurre de manera transparente en la capa del motor de base de datos, impidiendo que el cliente web altere los datos devueltos, independientemente del tipo de consulta realizada.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso Obligatorio de SCHEMABINDING:** Evita que las tablas o estructuras referenciadas en la función del filtro de seguridad sean alteradas o eliminadas físicamente de forma accidental, protegiendo la integridad del mecanismo de aislamiento.
- **Transparencia en el Desarrollo de Software:** La lógica de filtrado de registros se centraliza en la base de datos, liberando a los desarrolladores de la intranet de programar filtros de seguridad manuales en los controladores de Java.

PROYECTO 3: Enmascaramiento Dinámico de Datos (Dynamic Data Masking - DDM) para Datos Sensibles

1. Enunciado del ejercicio

Con el propósito de cumplir con la Ley de Protección de Datos Personales en el Perú, la universidad necesita proteger la información sensible de contacto de los alumnos expuesta en consultas ordinarias de reporte. Aplique técnicas de Enmascaramiento Dinámico de Datos (DDM) sobre las columnas Correo y Telefono de la tabla Academico.Estudiante, asegurando que los usuarios no administrativos solo visualicen cadenas formateadas o parciales.

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- 3.1. Aplicar enmascaramiento dinámico a la columna de Correo Electrónico (Formato de email integrado)
ALTER TABLE Academico.Estudiante
ALTER COLUMN Correo VARCHAR(100) MASKED WITH (FUNCTION = 'email()');
GO

-- 3.2. Aplicar enmascaramiento personalizado a la columna de Teléfono (Mostrar solo primer y último carácter)
ALTER TABLE Academico.Estudiante
ALTER COLUMN Telefono VARCHAR(20) MASKED WITH (FUNCTION = 'partial(2, "XXXXXX", 1)');
GO

PRINT 'Enmascaramiento dinámico de datos (DDM) inyectado en las columnas correspondientes.';
GO

```

3. Justificación técnica de la solución aplicada

Dynamic Data Masking opera en el flujo final de empaquetamiento del flujo de resultados (*Result Set*) devuelto por SQL Server. Los datos en el almacenamiento en disco se mantienen intactos, completamente legibles y procesables para operaciones de indexación y uniones. Sin embargo, si la sesión activa pertenece a un usuario sin el privilegio con el que cuenta el administrador (UNMASK), el motor reemplaza en caliente los caracteres lógicos por el patrón de enmascaramiento definido en los metadatos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Preservación del Rendimiento Relacional:** Al ejecutarse únicamente en la fase de presentación de resultados, no introduce penalizaciones de rendimiento en los planes de ejecución ni altera el funcionamiento de las restricciones de clave primaria o foránea.
- **Manejo Granular mediante Privilegios:** Permite otorgar el acceso a los datos limpios a usuarios específicos de auditoría mediante la concesión precisa del permiso estándar del sistema (GRANT UNMASK TO UsuarioAuditor).

PROYECTO 4: Script Automatizado para Respaldo Completo y Diferencial de Base de Datos

1. Enunciado del ejercicio

Diseñe un script administrativo automatizado para la UPLA que ejecute de manera secuencial la copia de seguridad integral del sistema de Grados y Títulos. El script debe generar un respaldo completo (Full Backup) al inicio de la jornada institucional y un respaldo diferencial (Differential Backup) al promediar la tarde, direccionando los archivos lógicos binarios comprimidos hacia la ruta asignada en el servidor (D:\BaseDatos2026\Backups\).

2. Script de la solución en T-SQL

```

USE master;
GO

-- 4.1. Generación del Respaldo Completo (Full Backup) de la Base de Datos
PRINT 'Iniciando copia de seguridad Completa...';
BACKUP DATABASE GradosTitulos
TO DISK = N'D:\BaseDatos2026\Backups\GradosTitulos_Full.bak'
WITH NOFORMAT, INIT,
     NAME = N'GradosTitulos-Copia Completa de Seguridad',
     SKIP, NOREWIND, NOUNLOAD,
     COMPRESSION, STATS = 10; -- Habilita la compresión nativa para ahorrar almacenamiento
GO

-- 4.2. Generación del Respaldo Diferencial (Differential Backup) de las transacciones acumuladas
PRINT 'Iniciando copia de seguridad Diferencial...';
BACKUP DATABASE GradosTitulos
TO DISK = N'D:\BaseDatos2026\Backups\GradosTitulos_Diff.bak'
WITH DIFFERENTIAL, NOFORMAT, INIT,
     NAME = N'GradosTitulos-Copia Diferencial de Seguridad',
     SKIP, NOREWIND, NOUNLOAD,
     COMPRESSION, STATS = 10;
GO

```

3. Justificación técnica de la solución aplicada

El respaldo completo captura la estructura DDL completa y todas las páginas lógicas de datos de la base de datos en un punto temporal exacto. El respaldo diferencial optimiza los recursos de infraestructura de la universidad al registrar únicamente los cambios y las extensiones de datos que sufrieron modificaciones desde el último respaldo completo. Incorporar la directiva **COMPRESSION** reduce sustancialmente el uso de operaciones de I/O en disco al procesar los archivos mediante algoritmos compactos antes de su escritura física.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso Clave de la Directiva INIT:** Sobrescribe de manera limpia cualquier contenido residual previo en el archivo destino físico, evitando el crecimiento descontrolado de los medios de almacenamiento y errores por espacio insuficiente.
- **Monitoreo de Progreso (STATS = 10):** Envía notificaciones periódicas cada 10 por ciento de avance al log de SQL Server, facilitando la auditoría de tiempos del proceso.

PROYECTO 5: Mantenimiento Preventivo: Reorganización y Reconstrucción de Índices Fragmentados

1. Enunciado del ejercicio

El constante flujo operativo de registro y emisión de resoluciones académicas genera fragmentación lógica en las estructuras internas de los índices de la base de datos, reduciendo el rendimiento general del motor. Desarrolle un script especializado de mantenimiento que evalúe y corrija este problema de forma automatizada: aplicando una reorganización en caliente si la fragmentación es moderada (entre 5% y 30%), o una reconstrucción total si supera el 30%.

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- SET NOCOUNT ON;

-- 5.1. Corregir fragmentación moderada mediante Reorganización (Operación en Línea ligera)
PRINT 'Ejecutando reorganización de índices en tablas de alta volatilidad...';
ALTER INDEX ALL ON Academico.Estudiante REORGANIZE;
ALTER INDEX ALL ON Tramite.Expediente REORGANIZE;
GO

-- 5.2. Corregir fragmentación crítica mediante Reconstrucción Completa (Reorganiza páginas físicas)
PRINT 'Ejecutando reconstrucción total para índices con fragmentación severa...';
ALTER INDEX ALL ON Documento.Diploma REBUILD WITH (ONLINE = ON, FILLFACTOR = 85);
ALTER INDEX ALL ON Evaluacion.Sustentacion REBUILD WITH (ONLINE = ON, FILLFACTOR = 85);
GO

```

3. Justificación técnica de la solución aplicada

La reorganización de índices (REORGANIZE) limpia las páginas de nivel hoja compactando los datos existentes a través del árbol de forma interna sin requerir grandes cantidades de memoria caché. La reconstrucción de índices (REBUILD) descarta por completo el árbol B+ obsoleto y fragmentado, creando una estructura lógica totalmente nueva y limpia en disco de forma contigua, lo que reduce las lecturas lógicas aleatorias y optimiza el uso del almacenamiento físico.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Disponibilidad Continua del Servicio (ONLINE = ON):** Permite reconstruir los índices pesados sin interrumpir los procesos de consulta o registro de los estudiantes, evitando bloqueos exclusivos en entornos de producción.
- **Preservación del Factor de Relleno (FILLFACTOR):** Al reconstruir, se vuelve a aplicar el espacio libre en las páginas, previniendo que futuras transacciones degraden rápidamente el rendimiento del servidor.

PROYECTO 6: Plan de Contingencia frente a Desastres: Restauración Completa de la Base de Datos

1. Enunciado del ejercicio

Ante un fallo crítico catastrófico de hardware que inhabilite el disco de almacenamiento del servidor principal de la UPLA, implemente un plan de contingencia de nivel de ingeniería que ejecute de forma ordenada la Restauración Completa de la base de datos GradosTitulos. El script debe garantizar la limpieza previa de cualquier conexión huérfana activa y levantar el sistema en un estado operativo consistente en una nueva unidad o servidor de contingencia.

2. Script de la solución en T-SQL

```

USE master;
GO

-- 6.1. Forzar el cierre inmediato de todas las sesiones de usuarios activas para liberar la base de datos
IF EXISTS (SELECT name FROM sys.databases WHERE name = N'GradosTitulos')
BEGIN
    ALTER DATABASE GradosTitulos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
END;
GO

-- 6.2. Ejecutar la secuencia de restauración completa del sistema
PRINT 'Iniciando proceso de restauración desde los medios de copias de seguridad...';
RESTORE DATABASE GradosTitulos
FROM DISK = N'D:\BaseDatos2026\Backups\GradosTitulos_Full.bak'
WITH REPLACE,          -- Sobrescribir de forma explícita la base de datos dañada existente
    RECOVERY,          -- Finalizar la transacción y dejar la base de datos en modo ONLINE apto para su uso
    STATS = 10;
GO

-- 6.3. Verificar y validar el estado operativo final del sistema restablecido
ALTER DATABASE GradosTitulos SET MULTI_USER;
PRINT 'El Plan de Contingencia concluyó exitosamente. Sistema GradosTitulos disponible.';
GO

```

3. Justificación técnica de la solución aplicada

La cláusula condicional inicial SINGLE_USER WITH ROLLBACK IMMEDIATE termina de forma segura cualquier conexión TCP persistente del backend de la aplicación, revirtiendo sus transacciones incompletas para desbloquear el control exclusivo del sistema de archivos. La opción REPLACE autoriza al motor a destruir los archivos de datos (.mdf) y logs (.ldf) corruptos anteriores, mientras que la directiva fundamental RECOVERY ejecuta la fase de reversión de transacciones (*Redo/Undo*), abriendo la base de datos con consistencia transaccional absoluta.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Garantía de Continuidad Operativa:** Provee un procedimiento documentado y determinista que reduce el Tiempo de Recuperación Objetivo (RTO) ante incidentes o desastres tecnológicos severos en la universidad.
- **Restauración del Acceso Multi-usuario:** Devuelve explícitamente el estado operacional a MULTI_USER al final de la restauración, permitiendo que la intranet universitaria restablezca sus servicios de manera automatizada.